

12-WEEK MLE INTERVIEW PREPARATION PLAN

31 August - 22 November 2026

240

FOCUSED HOURS

12

STRUCTURED WEEKS

5

WEEKLY TRACKS

Target: Machine Learning Engineer, SWE-ML, and Applied ML

DSA | Fundamentals | System Design | Implementation | Review

Completion rule: produce evidence. Watching or reading alone does not complete a block.

HOW TO USE THIS PLAN

1. Protect the weekly hours and choose the first block.
2. Work from a blank editor, whiteboard, or closed notes when recall is required.
3. Keep accepted code, notes, diagrams, tests, scores, and an error log.
4. Diagnose misses by cause, repair them, and schedule spaced re-solves.
5. Time the work and explain decisions aloud.

WEEKLY ALLOCATION

WEEK	DSA	FUND.	DESIGN	PRACTICE	REVIEW	TOTAL
1	8h	7h	2h	2h	1h	20h
2	8h	7h	2h	2h	1h	20h
3	8h	6h	3h	2h	1h	20h
4	8h	5h	3h	2h	2h	20h
5	7h	5h	4h	2h	2h	20h
6	7h	5h	4h	2h	2h	20h
7	7h	4h	4h	3h	2h	20h
8	7h	4h	4h	3h	2h	20h
9	6h	4h	5h	3h	2h	20h
10	5h	4h	5h	3h	3h	20h
11	5h	4h	5h	2h	4h	20h
12	4h	3h	4h	2h	7h	20h

WEEK 1

Foundations: execution workflow

31 Aug 2026 - 06 Sep 2026 | 20 focused hours

DSA - 8H

8 Easy: arrays, strings and hashing

1. Solve without reading solutions first: 8 Easy: arrays, strings and hashing.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 7H

Counting and probability foundations

1. Watch the matching TAAI lessons and attachments for: Counting and probability foundations.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 2H

Requirements, architecture and API design

1. Design on paper or a whiteboard: Requirements, architecture and API design.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

SQL fundamentals and NumPy shapes

1. Implement from a blank editor or SQL console: SQL fundamentals and NumPy shapes.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 1H

Unseen Easy assessment and error log

1. Run closed-book and timed: Unseen Easy assessment and error log.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 2

Probability and linear structures

07 Sep 2026 - 13 Sep 2026 | 20 focused hours

DSA - 8H

10: stacks, queues, linked lists and two pointers

1. Solve without reading solutions first: 10: stacks, queues, linked lists and two pointers.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 7H

Conditional probability, total probability and Bayes

1. Watch the matching TAAI lessons and attachments for: Conditional probability, total probability and Bayes.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 2H

Prediction-service API design

1. Design on paper or a whiteboard: Prediction-service API design.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

SQL joins, CASE and conditional aggregation

1. Implement from a blank editor or SQL console: SQL joins, CASE and conditional aggregation.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 1H

Re-solve assisted problems

1. Run closed-book and timed: Re-solve assisted problems.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 3

Trees and random variables

14 Sep 2026 - 20 Sep 2026 | 20 focused hours

DSA - 8H

10: recursion, trees, DFS, BFS and BST

1. Solve without reading solutions first: 10: recursion, trees, DFS, BFS and BST.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 6H

PMF, PDF, CDF and joint random variables

1. Watch the matching TAAI lessons and attachments for: PMF, PDF, CDF and joint random variables.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 3H

Caching, CDNs and invalidation

1. Design on paper or a whiteboard: Caching, CDNs and invalidation.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Implement variance, covariance and normalization

1. Implement from a blank editor or SQL console: Implement variance, covariance and normalization.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 1H

Tree-pattern recall check

1. Run closed-book and timed: Tree-pattern recall check.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 4

Windows, expectation and checkpoint one

21 Sep 2026 - 27 Sep 2026 | 20 focused hours

CHECKPOINT: Checkpoint 1

DSA - 8H

10: binary search, windows, pointers and prefix sums

1. Solve without reading solutions first: 10: binary search, windows, pointers and prefix sums.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 5H

Expectation, variance, covariance and distributions

1. Watch the matching TAAI lessons and attachments for: Expectation, variance, covariance and distributions.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 3H

Load balancing, consistent hashing and rate limiting

1. Design on paper or a whiteboard: Load balancing, consistent hashing and rate limiting.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Distribution and metric exercises

1. Implement from a blank editor or SQL console: Distribution and metric exercises.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

Checkpoint: coding, probability and mini-design

1. Run closed-book and timed: Checkpoint: coding, probability and mini-design.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 5

Graphs and sampling

28 Sep 2026 - 04 Oct 2026 | 20 focused hours

DSA - 7H

10: graph traversal, grids and components

1. Solve without reading solutions first: 10: graph traversal, grids and components.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 5H

Sampling distributions, CLT and sample variance

1. Watch the matching TAAI lessons and attachments for: Sampling distributions, CLT and sample variance.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 4H

SQL, NoSQL, replication, sharding and CAP

1. Design on paper or a whiteboard: SQL, NoSQL, replication, sharding and CAP.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Window functions and analytical SQL

1. Implement from a blank editor or SQL console: Window functions and analytical SQL.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

Graph and statistics revision

1. Run closed-book and timed: Graph and statistics revision.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 6

Heaps, backtracking and estimation

05 Oct 2026 - 11 Oct 2026 | 20 focused hours

DSA - 7H

10: heaps, intervals and backtracking

1. Solve without reading solutions first: 10: heaps, intervals and backtracking.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 5H

Point estimation, bias and confidence intervals

1. Watch the matching TAAI lessons and attachments for: Point estimation, bias and confidence intervals.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 4H

Queues, async workflows, MapReduce and TinyURL

1. Design on paper or a whiteboard: Queues, async workflows, MapReduce and TinyURL.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Implement splits, cross-validation and metrics

1. Implement from a blank editor or SQL console: Implement splits, cross-validation and metrics.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

Timed mixed set and estimator recall

1. Run closed-book and timed: Timed mixed set and estimator recall.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 7

Dynamic programming and experiments

12 Oct 2026 - 18 Oct 2026 | 20 focused hours

DSA - 7H

10: DP, knapsack and LCS

1. Solve without reading solutions first: 10: DP, knapsack and LCS.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 4H

Hypothesis tests, power and A/B experimentation

1. Watch the matching TAAI lessons and attachments for: Hypothesis tests, power and A/B experimentation.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 4H

ML design framework and spam/abuse system

1. Design on paper or a whiteboard: ML design framework and spam/abuse system.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 3H

Experiment SQL and metric implementation

1. Implement from a blank editor or SQL console: Experiment SQL and metric implementation.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

DP transitions and experiment review

1. Run closed-book and timed: DP transitions and experiment review.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 8

Linear models and checkpoint two

19 Oct 2026 - 25 Oct 2026 | 20 focused hours

CHECKPOINT: Checkpoint 2

DSA - 7H

8 hidden-topic Medium problems

1. Solve without reading solutions first: 8 hidden-topic Medium problems.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 4H

Linear/logistic regression, regularization and bias-variance

1. Watch the matching TAAI lessons and attachments for: Linear/logistic regression, regularization and bias-variance.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 4H

Rate limiter and TinyURL full designs

1. Design on paper or a whiteboard: Rate limiter and TinyURL full designs.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 3H

Implement linear and logistic regression

1. Implement from a blank editor or SQL console: Implement linear and logistic regression.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

Checkpoint: coding, ML fundamentals and design

1. Run closed-book and timed: Checkpoint: coding, ML fundamentals and design.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 9

Classical ML and recommendations

26 Oct 2026 - 01 Nov 2026 | 20 focused hours

DSA - 6H

8: trie, union-find, topo sort and shortest paths

1. Solve without reading solutions first: 8: trie, union-find, topo sort and shortest paths.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 4H

Trees, ensembles, SVM, KNN, clustering and evaluation

1. Watch the matching TAAI lessons and attachments for: Trees, ensembles, SVM, KNN, clustering and evaluation.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 5H

Recommendation-system design

1. Design on paper or a whiteboard: Recommendation-system design.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 3H

Ranking metrics, funnel and retention SQL

1. Implement from a blank editor or SQL console: Ranking metrics, funnel and retention SQL.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 2H

Mixed coding and model-selection review

1. Run closed-book and timed: Mixed coding and model-selection review.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 10

Linear algebra, neural nets and ranking

02 Nov 2026 - 08 Nov 2026 | 20 focused hours

DSA - 5H

6 timed mixed Medium problems

1. Solve without reading solutions first: 6 timed mixed Medium problems.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 4H

Projection, eigendecomposition, SVD and neural networks

1. Watch the matching TAAI lessons and attachments for: Projection, eigendecomposition, SVD and neural networks.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 5H

Search, ads or feed-ranking design

1. Design on paper or a whiteboard: Search, ads or feed-ranking design.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 3H

Dense layer, softmax, loss and backprop coding

1. Implement from a blank editor or SQL console: Dense layer, softmax, loss and backprop coding.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 3H

Coding and ML mock interviews

1. Run closed-book and timed: Coding and ML mock interviews.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 11

PCA, optimization and production ML

09 Nov 2026 - 15 Nov 2026 | 20 focused hours

DSA - 5H

5 new problems plus equal-time revision

1. Solve without reading solutions first: 5 new problems plus equal-time revision.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 4H

PCA, gradients, Jacobians and Hessians

1. Watch the matching TAAI lessons and attachments for: PCA, gradients, Jacobians and Hessians.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 5H

Production ML, drift, serving and fraud design

1. Design on paper or a whiteboard: Production ML, drift, serving and fraud design.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Matrix-calculus and PCA exercises

1. Implement from a blank editor or SQL console: Matrix-calculus and PCA exercises.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 4H

Three full mocks and targeted repair

1. Run closed-book and timed: Three full mocks and targeted repair.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.

WEEK 12

Interview simulation and consolidation

16 Nov 2026 - 22 Nov 2026 | 20 focused hours

CHECKPOINT: Final readiness audit

DSA - 4H

5 diagnostics and two coding mocks

1. Solve without reading solutions first: 5 diagnostics and two coding mocks.
2. For each problem record pattern, brute force, optimized approach, and time/space complexity.
3. Re-solve every hinted or failed problem from a blank editor after 24 hours.

EVIDENCE: Accepted code plus a solution log and failure-pattern note for every problem.

DONE WHEN: Target count accepted; each solution explained aloud; every miss scheduled for re-solve.

FUNDAMENTALS - 3H

Rapid oral ML and mathematics review

1. Watch the matching TAAI lessons and attachments for: Rapid oral ML and mathematics review.
2. Close the material and write a one-page recall sheet: definitions, equations, assumptions, and one worked example.
3. Complete attached questions and explain the topic aloud for five minutes without notes.

EVIDENCE: One recall sheet and worked answers to matching exercises.

DONE WHEN: You can derive the key result, answer exercises, and explain when the concept should not be used.

DESIGN - 4H

General and ML full-system designs

1. Design on paper or a whiteboard: General and ML full-system designs.
2. Write requirements, scale assumptions, API contracts, and data model.
3. Draw architecture; document bottlenecks, failure modes, observability, and two explicit trade-offs.

EVIDENCE: Reviewable diagram with APIs, storage, sizing, and trade-offs.

DONE WHEN: Present in 35 minutes and defend scaling, reliability, and ML decisions for 10 minutes.

PRACTICE - 2H

Final SQL and ML-coding assessment

1. Implement from a blank editor or SQL console: Final SQL and ML-coding assessment.
2. Add representative, boundary, and failure-case tests; run and fix every failure.
3. Save final work and write three bullets on complexity, correctness, and the main mistake avoided.

EVIDENCE: Executable code or SQL, tests, and a short engineering note.

DONE WHEN: Runs without copied solution code, passes tests, and every line and trade-off can be explained.

REVIEW - 7H

Final mocks, error-log closure and readiness audit

1. Run closed-book and timed: Final mocks, error-log closure and readiness audit.
2. Update the error log with cause, corrected mental model, and a prevention rule for every miss.
3. Repair weakest items and choose the first repair task for the next session.

EVIDENCE: Scored attempt, updated error log, and prioritized repair list.

DONE WHEN: Every miss has a diagnosed cause and scheduled repair; reading a solution never counts.